



Real-Time Error Handling Plan

How GetGuac detects, contains, surfaces, and recovers from errors as they happen — across the Flutter app, the Next.js web app, and Supabase.

Never crash the UI

Fail-open for ads

Fail-closed for money

Self-healing

1 • Principles

Every error-handling decision follows four rules. When in doubt, apply them in order.

- 1. Never crash the UI.** Any external call (network, store, SDK, DB) is wrapped; on failure the surface degrades to a calm empty/retry state, never a white screen or stack trace.
- 2. Fail open for ads.** An ad/entitlement read that errors must still let ads render and the page load. We never lose layout or revenue to a transient error.
- 3. Fail closed for money & access.** Premium, purchases, and any entitlement are granted only on *server-verified* proof. If we can't confirm, we don't grant. A client claim is never trusted.
- 4. Be idempotent & self-healing.** Operations can run twice safely (upserts, dedup, `completePurchase`); failed work retries automatically (outbox, `restorePurchases`, backoff) so a blip resolves without the user noticing.

2 • Error taxonomy & severity

Each error is classified by **category** (where) and **severity** (how urgent). Severity drives alerting and on-call response.

Category	Examples	Default severity	User-facing posture
Crash / unhandled	Null deref, render exception, uncaught async	S1	Caught globally; screen shows recover state, event uploaded
Payment / IAP	Verify fails, store unavailable, pending purchase	S2	Honest status + retry; never charge-without-grant
Auth / session	Expired token, refresh failure, RLS denied	S2	Silent refresh → re-login prompt if it fails
Server / API	5xx, timeout, dependency down (Gemini, SerpApi)	S2	Typed error + retry/backoff; fallback provider
AI parsing	Bad JSON, truncation, low confidence	S3	Fallback model, then manual-edit path
Network / offline	No connectivity, DNS, TLS	S3	Queue & retry (outbox); optimistic UI
Validation / input	Bad form value, oversized upload	S4	Inline field message, no server round-trip
Ads	No-fill, SDK init fail, blocked	S4	Slot collapses to zero height (fail-open)

S1 page/flow broken for many → alert now · **S2** a core flow degraded → alert in minutes · **S3** recoverable/handled → daily triage · **S4** expected/cosmetic → log only.

3 • Defense in depth — the three layers

Client (Flutter + Web)

- Global capture: `FlutterError.onError` + `PlatformDispatcher.onError` (mobile); error boundaries + Sentry (web)
- Every async call in try/catch; widgets degrade to empty/retry, never throw to the tree
- Retry with backoff for idempotent reads; optimistic UI with rollback for writes
- Offline queue (`receipt_outbox`) flushes when connectivity returns

API routes (Next.js)

- Typed status codes: 400 / 401 / 402 / 429 / 5xx — never a bare 500 for an expected case
- Input validated before side-effects; rate-limited (`apiGuard`)
- Idempotent writes (upsert on natural keys) so a client retry can't double-apply
- External deps (Gemini, SerpApi, Play API) wrapped with fallback + timeout

Database (Supabase / Postgres)

- Row-level security is the last line — a leaked bug still can't cross users
- Constraints + unique indexes turn races into safe conflicts (caught as upserts)
- Sensitive grants (`premium_entitlements`) are **service-role-write-only**

What we already run

- Mobile: `debug_log` 500-event ring (survives restart) → `client_logs`
- Mobile: `error_report_service` → `audit_log` (user "Report a problem")
- Web: Sentry; per-user `/api/me/logs` (RLS-gated, the user owns their logs)

4 • Real-time detection & telemetry

"Real-time" means we learn about an error within seconds, from the device, without asking the user to read a log.

Signal	Source	Where it lands	Latency
Uncaught exceptions / crashes	<code>onError</code> handlers (mobile), Sentry (web)	Sentry issue + <code>client_logs</code>	seconds
Flow breadcrumbs	<code>debug_log</code> ring buffer	<code>client_logs</code> on next session ping	seconds–minutes
User-reported problems	Profile → Report a problem	<code>audit_log</code> (action=error_report) + log tail	on submit
API failures	Route logs + status codes	Vercel logs / Sentry	seconds
Payment events	Play RTDN / Apple ASSN webhook	<code>iap_purchases</code> + entitlement	seconds

Gaps to close (see roadmap): add `global-error.js` for Sentry React-render capture on web; re-enable `sentry_flutter` for mobile crash grouping (currently a no-op stub due to an AGP/Kotlin compile issue); add Sentry **alert rules** (new-issue + crash-free-rate threshold) so detection becomes notification.

5 • Real-time user-response patterns

What the user actually sees, and what auto-recovery runs underneath, per category.

Situation	User sees	Under the hood
Transient network read	Skeleton, then content (or "Tap to retry")	2–3 retries w/ backoff; cached value if present
Receipt upload offline	"Saved — will sync" chip	<code>receipt_outbox</code> queues; flush on reconnect
AI parse fails	Receipt opens in editable state	Fallback model → manual fields; nothing lost
Session expired	Nothing (silent) → login if refresh fails	Token refresh; one re-auth prompt max
Purchase verify fails	"Activating your premium..." + Restore	Retry; <code>restorePurchases</code> re-verifies next launch
Ad no-fill / blocked	Nothing (no gap)	Slot collapses to zero height
Hard crash	Calm "Something went wrong — reload" card	Event captured; state restored where possible

6 • Flow playbooks (the money & capture paths)

6.1 In-app purchase → premium

- **Store unavailable / product missing** → `buyPremium()` returns false → Snackbar "not available". No dead-end.
- **Purchase succeeds, server verify fails** → **fail-closed**: premium not granted, but the purchase is re-delivered by the store, so verification retries. **Add**: retry+backoff on the verify call and a visible "activating..." state.
- **Pending purchase** (slow card / parental approval) → show a pending state; the stream fires again on resolution.
- **Duplicate delivery** → always `completePurchase` ; `iap_purchases` upsert on (platform, token) → idempotent.
- **Renewal / cancel / refund** → **must** be reconciled by a **Play RTDN / Apple ASSN webhook** that re-stamps `premium_until` . Without it, access goes stale.

6.2 Ads (web AdSense + native AdMob)

- Tier-1 always shows; Tier-2 hidden for premium. An entitlement-read error → **fail-open** → ads still show.
- AdSense unfilled / AdMob no-fill → the slot collapses (no reserved blank, no orphan label).
- SDK init failure (no app id) → ads simply stay off; the app is unaffected.

6.3 Receipt capture & parse

- Capture offline → outbox queue → flush on reconnect (no lost receipts).
- Parse error / bad JSON → fallback model, then a manual-edit receipt — the image is always kept.
- Duplicate (camera + email + statement of one purchase) → normalized-name + ±1¢ dedup collapses to one row.

7 • Alerting & escalation

Severity	Trigger	Channel	Response target
S1	Crash-free rate drops below threshold; new high-volume crash; entitlement-write failures	Push/page	Investigate immediately
S2	Spike in 5xx / verify failures / auth failures	Email/Slack	Within the hour
S3	Elevated handled errors (parse fallbacks, retries)	Daily digest	Next business day
S4	Expected/cosmetic (no-fill, validation)	Log only	Trend review

8 • Recovery & self-healing

- **Idempotency everywhere** — upserts on natural keys; `completePurchase` always called; dedup on capture.
- **Outbox flush** on app start + connectivity regain re-sends queued receipts.
- **restorePurchases** on launch re-verifies any unconfirmed entitlement.
- **Webhooks as source of truth** (RTDN/ASSN) keep premium accurate through renew/cancel/refund.
- **RLS as backstop** — even an unhandled bug can't leak another user's data.

9 • Implementation roadmap

Phase	Item	Why
Now	Add web <code>global-error.js</code> (Sentry React-render capture)	Today React render errors go uncaptured
Now	Sentry alert rules: new-issue + crash-free-rate	Turn detection into notification (S1/S2)
Next	IAP verify retry/backoff + "activating..." state + surface Restore on failure	Close the paid-but-not-granted gap
Next	Play RTDN / Apple ASSN webhook → re-stamp <code>premium_until</code>	Renewals / refunds / cancels stay accurate
Later	Re-enable <code>sentry_flutter</code> for mobile crash grouping	Currently a no-op stub (AGP/Kotlin issue)
Later	Health/uptime checks on key API routes + dependency status	Catch dependency outages before users do

Appendix • Where things live

Concern	Location
Mobile global capture	<code>main.dart</code> — <code>FlutterError.onError</code> + <code>PlatformDispatcher.onError</code>
Mobile diagnostic log	<code>services/debug_log.dart</code> → <code>client_logs</code>
User problem reports	<code>services/error_report_service.dart</code> → <code>audit_log</code>
Offline receipt queue	<code>services/receipt_outbox.dart</code>
Per-user log access	<code>/api/me/logs</code> (RLS-gated)
IAP verification	<code>/api/iap/verify</code> · <code>premium_entitlements</code> · <code>iap_purchases</code>
Web error monitoring	Sentry (@sentry/nextjs)